Gait Pattern Generation for Humanoid Robots using Reinforcement Learning

Dimitria Silveria
Department of Electrical & Computer Engineering
Queen's University
Kingston, Canada
23ldy@queensu.ca

Mateus Karvat Camara
School of Computing
Queen's University
Kingston, Canada
mateus.karvat@queensu.ca

***Abstract*—One of the biggest challenges in the field of humanoid robots is walking because it involves two stages: gait pattern generation and trajectory following control. Most of the classical approaches use dynamic models, such as pendulums, to generate curves that represent the robot's center of mass movements along with controllers to follow these curves. However, these approaches require parameter tuning for both stages. Our approach aims to surpass this issue for the first stage, by using a Reinforcement Learning agent to learn trajectories that produce human-like walk and, at the same time, are feasible for the controllers. We used the SAC algorithm with a Multilayer Perceptron as the agent and designed a Gymnasium environment with the NAO robot. We also used the PyBullet simulator to train our model and visualize the results. We realized that the robot successfully learned how to walk, but the model can still be improved to generate a straight-line trajectory and be generalizable to other humanoid robots.**

***Index Terms*—Reinforcement Learning, Robotics, Humanoid robots, NAO Robot, Gait pattern**

I. Introduction

The RoboCup [1] is an international robotics competition which aims to foment advances in robotics technologies. One of its leagues is the Humanoid RoboCupSoccer, in which humanoid robots play soccer against each other. Many features must be implemented on the robots to accomplish this task, such as object detection, walking, navigation, strategies, and others. Among all of them, robot walking is a significantly challenging field of robotics because it involves gait pattern generation and trajectory following control, which are two challenging fields in themselves.

The work from Silveria [2] proposes a gait control pipeline focused on the RoboCup context. In this work, the Linear Inverted Pendulum Model (LIPM) [3] generates the robot's center of mass (CoM) trajectory and a Linear Quadratic Regulator (LQR) controls this trajectory. For the swinging leg trajectory, Bezier [4] curves were used as input to a Proportional controller with Feed-Forward. Although the LIPM model is largely used for humanoid robots' gait generation, it presents two downsides. The first one is the assumption that the z coordinate (which corresponds to the CoM's height) is constant throughout the trajectory, which was shown not to be true [2]. The other one is that the LIPM's paper [3] doesn't provide any insight into selecting some of the model's parameters, such as the pendulum length and the robot's step period. As such, the process of selecting these problems, for a given robot, becomes a time-consuming trial-and-error process. Another drawback of this work is that it only generates straight-line trajectories, but during a soccer match, the robot needs to execute much more complex ones.

Therefore, to overcome the the flaws found in Silveria's work [2], we expand upon it by replacing the LIPM with a Reinforcement Learning (RL) agent that generates parameters for sinusoidal curves which can be used to describe the robot's CoM and swinging foot trajectories. These trajectories are inputs for LQR and Proportional controllers which generate, respectively, the supporting leg and swinging leg angles, that are then executed by a simulator to emulate real-world physics. The whole pipeline of our work is displayed in Figure 1.

Our approach tackles both gait pattern generation as well as trajectory following issues because our agent can learn the gate patterns that are feasible for specific controllers' gains. Besides that, it also allows the generation of more complex gait patterns by changing the trajectory input functions.

II. Related Work

The model proposed by Kajita *et al.* [3], the Linear Inverted Pendulum Model (LIPM), assumes that an inverted pendulum can represent the dynamics of the Center of Mass (CoM), and the trajectory remains at a constant height, $z = z_c$. It is widely used for gait pattern generation because it is computationally "cheap", due to the constant height assumption, and can be used for any type of humanoid robot. However, one disadvantage of this model is that it doesn't have any process to find the model's parameters, such as pendulum length, gait period, robot step size, and speed. This way it is up to the designer to guess which ones are the best for a specific robot. This guessing process may be time-consuming and there is no guarantee the parameters will be optimal, which makes the LIPM hard to implement, in terms of design.

Aiming to surpass the aforementioned limitations, the work presented in [5] proposes the use of a dynamic model capable of generating trajectories close to human walking, called *3D Dual Spring-Load Inverted Pendulum (Dual-SLIP)*. This model considers the CoM as an inverted pendulum with double springs. The springs are used to model the movement in the z

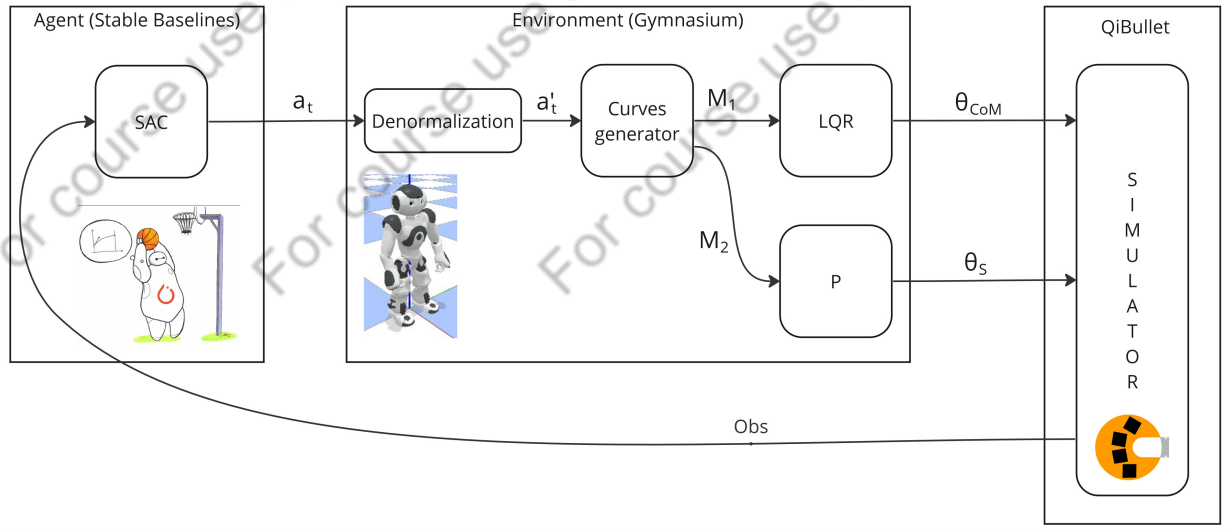


Fig. 1: Pipeline for the proposed implementation. The SAC agent learns parameters for sinusoidal curves that are used by our environment to generate joint angles sent to the simulator. After the simulated NAO robot has performed one step, the simulator returns the observation values. Thus, each step of the robot corresponds to one step of the agent and each training episode runs until the robot falls or until the maximum episode length is reached.

axis. It also proposes an optimization process to calculate its parameters.

Despite being robust, Dual-SLIP has a limitation related to the Froude [6] number, defined as $Fr = v^2/gl$, where v is the walking speed, g is the gravitational acceleration and l is the length of the legs. It is only capable of generating human-like walks when this number is within the range $0.02 - 0.22$ [5]. However for some humanoids, such as NAO, whose maximum speed is 14 cm/s and $l \approx 0.21$, its maximum value is ≈ 0.01.

We want to overcome these two models' limitations by using Reinforcement Learning in the process of gait pattern generation. The idea is for the agent to learn the patterns for any humanoid robot, surpassing the Dual-SLIP generalization issue, with movements in the three axes, avoiding the guessing process and z coordinate limitation of the LIPM. Besides that, our agent is not limited to learning the CoM trajectory, but also the swinging foot movements, which provides an extra shortcut in the design process.

Apart from trajectory generation, trajectory following is another issue in the humanoid walk. To tackle this problem, [2] proposed the use of an LQR controller for the CoM, which would output the supporting leg angles, and a proportional controller with feedforward to output the swinging foot, angles, both using dual-quaternion algebra. This approach works well once the trajectory is known and the controller gains are properly tuned for it. However, re-tuning the controller for each unique pattern is time-consuming and may be undesirable in applications that require the controlled agent to execute trajectories that were not previously known. Aiming to solve this problem, [7] proposes the combination of a Deep Neural Network (DNN) and PID architectures for quadrators trajectory following. Despite the good results, this approach still requires training data for the DNNs. To avoid that, we expect our RL agent to be able to learn trajectories that are feasible for the controller, instead of "teaching" our controller to adapt to its inputs.

III. Methodology

According to the pipeline shown in Figure 1, we used a Soft-Actor-Critic (SAC) as an agent with a Multilayer Perceptron (MLP) as neural network architecture, with two hidden layers with 512 neurons each. All of them are provided by the Stable Baselines [8] library. We also designed our environment with Gymnasium [9], using the PyBullet simulator [10] to emulate the physics along with the library QiBullet [11], which provides the NAO robot object.

A. Action Space, Observation Space, and Reward

By analyzing the CoM and swinging foot trajectories in [5], we realized that they were periodic curves that sinusoidal functions could approximate. Consider a reference frame $O_I = \{X_I, Y_I, Z_I\}$, with axes X_I and Y_I defining the horizontal plane and Z_I pointing upwards. Also, x, y, and z are position values along the axes X_I, Y_I, and Z_I. We consider the robot to be moving along the X_I axis. This way, we have 3 sinusoidal curves (represented as $f()$):

- $z = f_1(x)$, for the CoM, from 0 to 2π and phase approximately π,
- $y = f_2(x)$, for the CoM, from 0 to π and phase approximately π when the left leg was swinging and with phase approximately 0 otherwise,
- $z = f_3(x)$, for the swinging foot, from 0 to π and phase 0.

Therefore, we defined our action space as:

$$action = [a_1, a_2, a_3, a_4, a_5, a_6, a_7] \in \mathbb{R}^7 \quad (1)$$

where each $a_i \in [-1, 1]$ corresponds to one of the sinusoid's parameters (amplitude, frequency or phase), which will be discussed with more detail in the Subsection III-B.

Our first idea was to return the positions x, y, and z of the CoM and both feet as observations, but we figured that the joint angles would provide more important information for the agent because it would give it better insight into what each controller turns the actions into. Therefore, the observations, returned by the simulation, were defined according to the equation:

$$Obs = [\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6, \theta_7, \theta_8, \theta_9, \theta_{10}, \theta_{11}, \theta_{12}] \in \mathbb{R}^{12}, \tag{2}$$

in which each θ_i corresponds to one of the legs joint angles at the end of each step. By using this observation, we wanted the agent to associate which parameters of the sinusoid would lead the controllers to output angles that generated a walk pattern, and which ones would cause the robot to fall.

As for the reward function, we were planning on using $1/t$, in which t is the total time from the beginning of the simulation so far. However, after a few experiments, we realized the agent was practically marching in place, to execute the steps faster, thus minimizing the time, which defeats our goal.

Therefore, the rewards were returned according to the equation:

$$Reward = \Delta x + \frac{1}{n}, \tag{3}$$

in which Δx is the distance that the robot walked for that particular step, in the x-axis, and n is the current number of steps. We chose this reward because we wanted the robot to prioritize the number of steps against the walked distance at the beginning, causing the first steps to be small and leading to a more stable start of its walk, according to [3], decreasing thus, the chances of falling at the beginning. Then, we wanted the agent to shift this priority as the number of steps increases, to encourage larger steps to be taken later on.

B. Environment

In our formulation, each step of the RL agent corresponds to a step of the robot. Thus, each action-observation pair corresponded to the parameters of one robot step.

The first stage of the environment, after receiving the actions, is the denormalization of the actions. This is an important stage because, as we mention in Section II, we want to overcome the generalization problem. To do that, our agent selects normalized actions, and this stage is responsible for adjusting them according to the robot's parameters, such as links' lengths. Later on, the trajectories are generated according to the following equation:

$$\begin{aligned} z_{CoM} &= a_1 * \sin a_2 t + a_3 \\ y_{CoM} &= a_4 * \sin a_2 t + a_5 \\ z_{foot} &= a_6 * \sin a_7 t. \end{aligned} \tag{4}$$

The curves for z_{CoM} and y_{CoM} (which are represented by M_1 in the Figure 1) are used as inputs to the LQR controller, which turns them into the supporting leg joint angles, (θ_{CoM} in the Figure 1), and z_{foot} (which is represented by M_2 in the Figure 1) is the input to the proportional controller with feed-forward, which generates the swinging foot angles (θ_s in the Figure 1). At the end of each step, the final joint angles are collected from the simulator and sent to the agent as observation. In addition to that, we collect the absolute distance in the x-axis the robot has walked so far to calculate our rewards and to improve the evaluation of our results.

IV. Controllers

The controllers' gains were selected by manually inputting some sinusoidal curves that yielded gait patterns, for the NAO robot, and adjusting them to follow these trajectories. Figure 2 shows a block diagram for the LQR controller. The input is the error between the reference trajectories ($z_{ref}(x)$ and $y_{ref}(x)$), and the actual trajectories ($z_{ref}(x)$ and $y_{ref}(x)$). The controller actuates by calculating the derivatives of the desired supporting leg angles ($\dot{\vec{\theta}}_d$) that make the error go to zero. These derivatives are integrated, to obtain the desired angles ($\vec{\theta}_d$), which are executed by the simulator, which also outputs the actual angles. The actual trajectories are, then, calculated by the robot's forward kinematics (FKM).

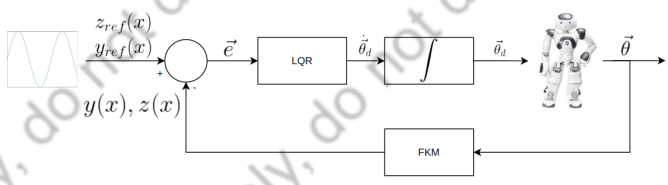


Fig. 2: LQR controller for the Center of Mass.

Figure 3 represents the block diagram for the proportional controller with feed-forward (K+FW). We can see that it does something similar to the LQR, except that it calculates the angles for the swinging foot.

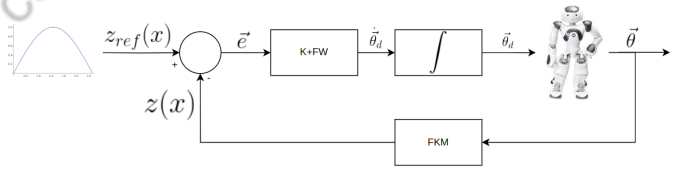


Fig. 3: Proportional controller with Feed-Forward for the swinging foot.

The mathematical formulation for both controllers is discussed in more detail by [12].

V. Results and Discussion

By implementing the agent and environment described in Section III, we ran several experiments in which the agent was incapable of learning the proper parameters for the sinusoidal curves that would allow it to walk more than 2 steps. For most of these experiments, the robot would fall after a single step. The experiments and the main reason for their shortcomings

are described in Subsection V-A. After finding out the main issue, we focused on two experimental scenarios: having an action space of $\mathbb{R}^2$ or an action space of $\mathbb{R}^7$. Subsection V-B presents the results of these final experiments and discusses their results.

A. Preliminary experiments

Over 100 experiments, accounting for over 20 million training episodes, were performed, in which diverse training strategies were attempted, such as:

- Reward shaping
  - Absolute x value of the robot at each step
  - Δx of the robot at each step
  - Inclination of the robot in regards to the z axis
  - Total number of steps
- Using PPO instead of SAC
- Changing the size of the action space from $\mathbb{R}^7$ to $\mathbb{R}^2$
- Changing the range of values for action space and observation space parameters (e.g. $[0, 1]$, $[-1, 1]$, etc.)
- Discretizing actions and observations
- Using a single sinusoidal curve for the whole walk instead of one curve for each step
- Using Float64 or Float32

After performing all of these experiments, it was clear that the issue was not related to any of the training parameters, which prompted a careful investigation of the actions being performed by the robot within the environment during the learning process, which pointed out the main issue behind the failure of all of these experiments: the simulator, when resetting the robot to its initial state, maintained its angular positions, its angular speed and never positioned the robot precisely in the same location as it was originally. Therefore, we understood it was necessary to explicitly reset the robot's angles after resetting the simulation and account, in the learning process, for varying initial states.

B. Final experiments

By tackling the simulator issue, we were able to train the agent to successively choose actions that enabled the simulated NAO robot to walk multiple steps. Then, all of the different experimental conditions mentioned in Subsection V-A were now viable experimental settings, for which we chose the one which we considered the most significant: the change in action space from $\mathbb{R}^7$ to $\mathbb{R}^2$. In the $\mathbb{R}^7$ action space, the agent chooses all of the values shown in Equations 1 and 4, while in $\mathbb{R}^2$ it chooses only a_2 and a_7, with the remaining values being chosen from an approximation of the curves generated by the LIPM model:

$$\begin{aligned} a_1 &= -0.0001 \\ a_3 &= 0 \\ a_4 &= 0.015 \\ a_5 &= 0 \\ a_6 &= 0.015 \end{aligned} \tag{5}$$

These two values that the agent chooses, a_2 and a_7, correspond, prospectively, to the CoM's and swinging foot traveled distances in the x axis. They are the only two parameters that vary for each step, in the LIPM model. Therefore, we decided to use all the parameters given by the LIPM (except for the z_{CoM} amplitude, which corresponds to a_1, which we selected by trial and error), an let our agent pick the remaining ones.

The results of these experiments are shown in Figure 4, which shows that the results for the $\mathbb{R}^2$ experiment were better than those for the $\mathbb{R}^7$ experiment. From Subfigure 4a, it is clear that, while the $\mathbb{R}^2$ experiment converged to the maximum episode length of 100 timesteps, the $\mathbb{R}^7$ one not only did not converge to the maximum length, but it did not converge at all. Then, from Subfigure 4b, the effects of that are apparent, where the distance walked by the robot in the $\mathbb{R}^2$ action space was always greater than 2.0 meters after 100k episodes, while the $\mathbb{R}^7$ action space barely reached 1.5 meters. The results for $\mathbb{R}^2$ were better because the agent had to select only two actions, which would lead it to faster convergence than by selecting 7. Besides that, we had the guarantee, by the LIPM model, that the other 5 parameters would work.

Finally, Subfigure 4c shows that the reward function manages to capture the desired aspects of the robot walk by considering both the total number of steps taken in an episode as the overall distance traversed by the robot in the x axis.

Given the superior results of the $\mathbb{R}^2$ experiment, we performed the model prediction (with Stable Baseline's [8], `model.predict()`), which are shown in Figure 5. We can see that, for the best policy for that experiment, the agent was able to iteratively select values that allowed the robot to walk more than 3 meters. However, by plotting the x, y position of the center of mass of the robot throughout that walk, we have Figure 6, which shows that the trajectory followed by the robot was not a straight line.

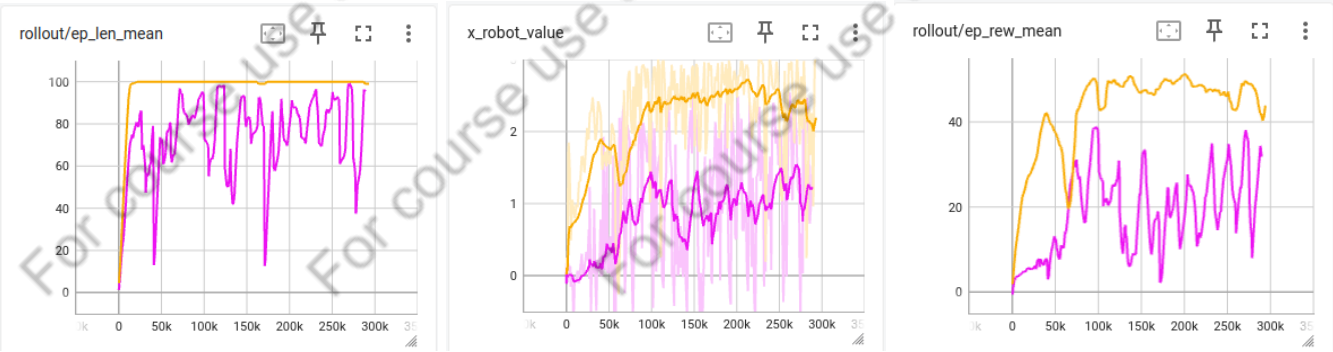
VI. Conclusion

We have successfully implemented a Reinforcement Learning agent that learns the parameters necessary to generate sinusoidal curves of a humanoid robot's Center of Mass and swinging foot, which are used by an LQR and a Proportional controller to generate the angles required for the robot to perform successive steps and thus, walk.

As it stands, our project failed to overcome the limitations of the Linear Inverted Pendulum Model, mainly its time-consuming design, since our implementation also requires the parameters from Equation 5 to be manually defined by the designer. However, such limitation might be overcome by making improvements to the $\mathbb{R}^7$ action space so that it achieves better results than the ones achieved so far by the $\mathbb{R}^2$ action space.

Moreover, several changes can still be taken to improve the robot's trajectory, overall distance and the generalizability of the agent, such as:

- Adding yaw to the observation and rewards to aid the robot in walking in a straight line;
- Implementing normalization of the angles observation so that the agent can be generalized to other robots with different angle configurations;



(a) Average episode length (b) Distance walked in the x axis (c) Average reward

Fig. 4: Comparison of results for the $\mathbb{R}^2$ (yellow) and the $\mathbb{R}^7$ (pink) action spaces throughout training. Maximum episode length was set to 100 timesteps. A smoothing factor of 0.9 is used for the distance plot.

```
Step (10): x_robot_value: 0.259, reward: 0.676
Step (20): x_robot_value: 0.576, reward: 0.605
Step (30): x_robot_value: 0.893, reward: 0.57
Step (40): x_robot_value: 1.197, reward: 0.539
Step (50): x_robot_value: 1.509, reward: 0.507
Step (60): x_robot_value: 1.808, reward: 0.534
Step (70): x_robot_value: 2.132, reward: 0.525
Step (80): x_robot_value: 2.443, reward: 0.521
Step (90): x_robot_value: 2.757, reward: 0.526
Step (100): x_robot_value: 3.063, reward: 0.541
Total reward: [57.37053]. Time: 235.088
```

Fig. 5: Results for the best policy after training. A total distance of 3.063 meters was walked in the x axis, resulting in a cumulative reward of 57.37053.

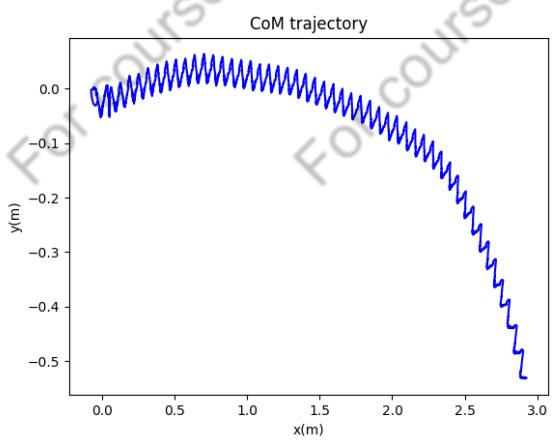


Fig. 6: Trajectory of the center of mass of the robot along the x and y axis by following the best policy. Despite going as far as 3 meters in the x axis, the robot's trajectory is curved and it moves 0.5 meters in the y axis.

- Increasing the maximum length of episodes and training the agent longer;
- Modifying the Multilayer Perceptrons used by the SAC algorithm;
- Working with a different simulator.

ACKNOWLEDGEMENTS

We would like to acknowledge the help and counselling of members of the QUARRG laboratory, from Queen's University, especially Dr. Kleber Cabral.

REFERENCES

[1] RoboCup, "RoboCupSoccer - Humanoid," https://www.robocup.org/leagues/3, Agosto 2022.
[2] D. Silveria, "Controle de caminhada para robôs humanóides kidsize," 2022, Projeto Final de Curso.
[3] S. Kajita, F. Kanehiro, K. Kaneko, kiyoslii Fujiwara, K. Yokoi, and H. Hirukawa, "A realtime pattern generator for biped walking," *IEEE International Conference on Robotics & Automation*, vol. 1, no. 4, pp. 31–37, 2002.
[4] F. C. Machado, "Curvas de bézier e desenho de fontes tipográficas," https://www.ime.usp.br/ fabcm/files/projeto_bezier.pdf, Agosto 2022.
[5] Y. Liu, "A dual-slip model for dynamic walking in a humanoid over uneven terrain," Ph.D. dissertation, The Ohio State University, 2015, unpublished thesis.
[6] M. J. O. Christopher L Vaughan, "Froude and the contribution of naval architecture to our understanding of bipedal locomotion," *PubMed.gov*, vol. 1, no. 12, pp. 33–48, 2019.
[7] Q. Li, J. Qian, Z. Zhu, X. Bao, M. K. Helwa, and A. P. Schoellig, "Deep neural networks for improved, impromptu trajectory tracking of quadrotors," *CoRR*, vol. abs/1610.06283, 2016. [Online]. Available: http://arxiv.org/abs/1610.06283
[8] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-1364.html
[9] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. d. Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG, M. Krimmel, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, A. T. J. Shen, and O. G. Younis, "Gymnasium," Mar. 2023. [Online]. Available: https://zenodo.org/record/8127025
[10] K. Greff, F. Belletti, L. Beyer, C. Doersch, Y. Du, D. Duckworth, D. J. Fleet, D. Gnanapragasam, F. Golemo, C. Herrmann, T. Kipf, A. Kundu, D. Lagun, I. Laradji, Hsueh-Ti, Liu, H. Meyer, Y. Miao, D. Nowrouzezahrai, C. Oztireli, E. Pot, N. Radwan, D. Rebain, S. Sabour, M. S. M. Sajjadi, M. Sela, V. Sitzmann, A. Stone, D. Sun, S. Vora, Z. Wang, T. Wu, K. M. Yi, F. Zhong, and A. Tagliasacchi, "Kubric: A scalable dataset generator," 2022.

[11] M. Busy and M. Caniot, "qiBullet, a Bullet-based simulator for the Pepper and NAO robots," *arXiv preprint arXiv:1909.00779*, 2019.
[12] B. V. Adorno, "Two-arm manipulation: From manipulators to enhanced human-robot collaboration," Ph.D. dissertation, Université Montpellier II - Sciences et Techniques du Languedoc, 2011, unpublished thesis.